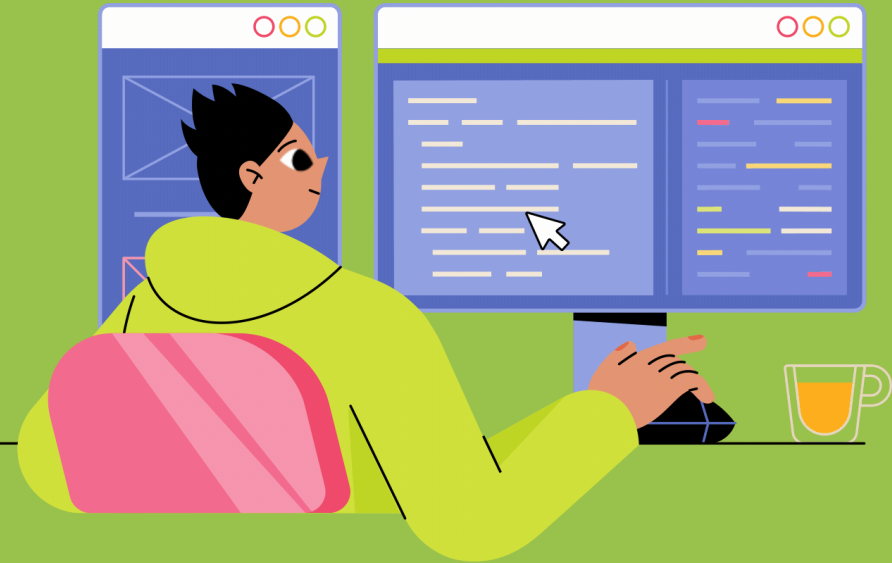




Node.js #2

Ratiporn Chanklan

หัวข้อ

- การติดตั้ง Node.js
- คลาส
- โมดูล
- การสร้างเว็บเซิร์ฟเวอร์ด้วย Node.js
- การใช้ Express.js ในการสร้าง Web Server
- การอ่านเนื้อหาจากไฟล์



npm.ps1 cannot be loaded

- เปิด PowerShell แล้วพิมพ์:

Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

- ทดสอบอีกครั้ง

node -v

npm -v

```
PS C:\Users\Lenovo\Desktop\test2> npm -v
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see
about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm -v
+ ~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

```
PS C:\Users\Lenovo\Desktop\test2> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\Users\Lenovo\Desktop\test2> npm -v
10.9.3
```



HTTP methods

- HTTP methods คือชุดของคำสั่งที่ใช้ในการกำหนดการกระทำที่เกิดขึ้นบนเซิร์ฟเวอร์เมื่อได้รับ Request จาก Client ผ่านโปรโตคอล HTTP โดยมีหน้าที่เป็นตัวกำหนดการกระทำต่างๆ ที่เกี่ยวข้องกับทรัพยากรบนเว็บเซิร์ฟเวอร์
- เมธอดที่พบบ่อยจะมีอยู่ด้วยกัน 4 ตัว:
 - 1.GET: ใช้สำหรับขอข้อมูลจากเซิร์ฟเวอร์ โดยไม่มีการเปลี่ยนแปลงข้อมูลที่อยู่บนเซิร์ฟเวอร์ เช่น การเรียกดูเพจเว็บหน้าแรก การขอข้อมูลรายการสินค้า เป็นต้น
 - 2.POST: ใช้สำหรับส่งข้อมูลจาก client ไปยังเซิร์ฟเวอร์เพื่อสร้างหรืออัปเดตข้อมูล โดยทั่วไปใช้สำหรับการสร้างข้อมูลใหม่ เช่นการสร้างบัญชีผู้ใช้ใหม่ เป็นต้น
 - 3.PUT: ใช้สำหรับการอัปเดตข้อมูลที่มีอยู่บนเซิร์ฟเวอร์ เช่นการอัปเดตข้อมูลโปรไฟล์ผู้ใช้ การแก้ไขข้อมูลสินค้าในระบบ เป็นต้น
 - 4.DELETE: ใช้สำหรับลบข้อมูลที่มีอยู่บนเซิร์ฟเวอร์ เช่นการลบโพสต์จากบล็อก การลบสินค้าออกจากระบบ เป็นต้น

Routing ใน Express (Express Routing)

- Routing (เส้นทาง) คือการกำหนดว่า เมื่อมี Request เข้ามาที่ URL และ Method ที่กำหนด → จะให้ Handler (โค้ด) ทำงานอะไร
- Route Methods ประกอบไปด้วย GET, POST, PUT, PATCH, DELETE
- รูปแบบการกำหนด Route

app.METHOD(PATH, HANDLER)

- โดย
 - **app** คือ instance ของ express (เข้าใจง่ายๆ "ฉั้น (app)" คือตัวแทนของ express)
 - **METHOD** คือ HTTP request method ใช้เป็นตัวเล็ก หมายถึง ทำการเรียกข้อมูลผ่าน HTTP ด้วยวิธี * get | post | put | delete
 - **PATH** คือ URL หรือ ส่วนของโดเมนนับตั้งแต่ "/" ตัวที่ 3 เป็นต้นไป เช่น *http://niik.in/forum จะได้ PATH คือ "/fourn" ถ้าเป็น Root PATH ก็คือ "/"
 - **HANDLER** คือฟังก์ชันที่กำหนดให้ทำงาน เมื่อ PATH ตรงกับค่าที่กำหนด



การใช้ Express.js ในการสร้าง Web Server

- 1. สร้างโฟลเดอร์เก็บงาน
- 2. เปิด VS code เข้าถึงโฟลเดอร์เก็บงาน
- 3. สร้างไฟล์หลัก เช่น app.js, index.js
- 4. เปิด terminal ใช้คำสั่ง npm init -y เพื่อสร้างไฟล์ package.json
- 5. เปิดไฟล์ package.json แล้วเพิ่ม "type": "module" และ กำหนดไฟล์เริ่มต้นของโปรเจกต์ที่ "main"
- 6. ติดตั้ง Express.js ได้ผ่านคำสั่ง npm install express
- 7. เพิ่มโค้ดสร้าง web server ใน ไฟล์หลัก เช่น app.js, index.js
- 8. ติดตั้ง nodemon โดยใช้คำสั่ง npm install nodemon
- 9. เปิด server โดยใช้ npx nodemon index.js

```
import express from "express";
const app = express();
const PORT = 3000;

// GET / → ตอบ Hello
app.get("/", (req, res) => {
  res.status(200).type('text/plain').send("Hello, Home Page");
});

// GET /about → ตอบ About
app.get("/about", (req, res) => {
  res.status(200).type('text/plain').send("This is the About page");
});

app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

สามารถกำหนด scripts ในไฟล์ package.json

- ในไฟล์ package.json คุณสามารถกำหนด scripts เพื่อระบุคำสั่งที่สามารถรันได้ง่ายๆ ผ่าน NPM
- เช่น ทดลองกำหนด `"start": "node index.js"`
 - เป็นคำสั่งเริ่มต้นที่สามารถรันได้ด้วย `npm run start` เพื่อ รันไฟล์ `index.js`

```
{  
  "name": "nodejs",  
  "version": "1.0.0",  
  "main": "index.js",  
  "type": "module",  
  "scripts": {  
    "test": "echo \\\"Error: no test specified\\\" && exit 1",  
    "start": "node index.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": "",  
  "dependencies": {  
    "express": "^4.21.2"  
  }  
}
```



การกำหนดพารามิเตอร์ของ Routing (Route Parameters)

- Route Parameters

- คือการกำหนด ค่าตัวแปรใน URL โดยใช้เครื่องหมาย :
- Express จะเก็บค่าที่ส่งมาผ่าน URL ไว้ใน req.params
- เช่น

```
app.get("/users/:id", (req, res) => {  
  const { id } = req.params; // หรือ const id = req.params.id;  
  res.send(`User ID: ${id}`);  
});
```

- สมมติถ้าเรียก GET /users/123
- จะได้ req.params = { id: "123" }
- ในโค้ด const { id } = req.params; เท่ากับเขียนว่า: const id = req.params.id;



การกำหนดพารามิเตอร์ของ Routing (Route Parameters)

- Route Parameters กรณีมีพารามิเตอร์หลายตัว

- เช่น

```
app.get("/products/:category/:id", (req, res) => {  
  const { category, id } = req.params;  
  res.send(`Category: ${category}, Product ID: ${id}`);  
});
```

- สมมติถ้าเรียก GET `http://localhost:3000/products/books/55`
- จะได้ `req.params = {
 category: "books",
 id: "55"
}`



การตรวจสอบ URL และ Query String

- Express มี properties หลายตัวใน req ที่ใช้ได้

- req.hostname เก็บค่า ชื่อโฮสต์ (domain หรือ IP) เช่น localhost หรือ example.com
- req.originalUrl เก็บค่า URL เต็ม (path + query string) เช่น /products/books?id=55&page=2
- req.path เก็บค่า path (ไม่รวม query) เช่น /products/books
- req.protocol เก็บค่า โพรโทคอล เช่น http หรือ https
- req.query เก็บค่า Query string คือส่วนของ URL ที่อยู่หลังเครื่องหมาย ? ใช้สำหรับส่งข้อมูลแบบ key=value ไปพร้อมกับ request อยู่ต่อท้าย path ของ URL สามารถมีได้หลายค่า โดยคั่นด้วย &
 - เช่น <http://localhost:3000/products/books/55?page=2>
 - http = protocol
 - localhost:3000 = hostname + port
 - /products/books = path
 - ? page=2 = query string (มีพารามิเตอร์ 1 ตัวคือ page)

```
app.get("/products/:category/:id", (req, res) => {  
  console.log("Hostname:", req.hostname);  
  console.log("Full URL:", req.originalUrl);  
  console.log("Path only:", req.path);  
  console.log("Protocol:", req.protocol);  
  console.log("Query.page :", req.query.page );  
  res.send("Check your console log!");  
});
```

```
[nodemon] starting `node index.js`  
http://localhost:3000  
[Mon Sep 08 2025 22:46:55 GMT+0700 (เวลาอินโดจีน)] GET /products/books/55?page=2  
Hostname: localhost  
Full URL: /products/books/55?page=2  
Path only: /products/books/55  
Protocol: http  
Query.page : 2
```



การเปลี่ยนเส้นทางด้วย Redirect()

- Redirect (การเปลี่ยนเส้นทาง) เป็นอีกความสามารถหนึ่งของ Express ที่ใช้บ่อยเวลาต้องพาผู้ใช้ไปยัง URL อื่น
- มีรูปแบบ `res.redirect([statusCode], path)`
 - statusCode → ค่า HTTP status code (ค่า default = 302 Found = redirect ชั่วคราว)
 - 301 Moved Permanently → redirect แบบถาวร (เช่น เปลี่ยน domain)
 - 302 Found (default) → redirect ชั่วคราว
 - 307 / 308 → redirect แบบชั่วคราว/ถาวร แต่ยังคง method และ body เดิม
 - path → URL ปลายทางที่จะส่งผู้ใช้ไป

```
// Redirect แบบถาวร
app.get("/old-page", (req, res) => {
  res.redirect(301, "/new-page");
});

app.get("/new-page", (req, res) => {
  res.send("นี่คือหน้าใหม่ (Permanent Redirect)");
});
```



Middleware คืออะไร?

- Middleware คือฟังก์ชันที่ทำงาน “คั่นกลาง” ระหว่าง Request กับ Response
 - สามารถอ่าน/แก้ไข req และ res ได้
 - ใช้ตรวจสอบสิทธิ์, แปลงข้อมูล, log, จัดการ error ฯลฯ
 - ต้องมีพารามิเตอร์ (req, res, next)

```
import express from "express";
const app = express();
const PORT = 3000;

// middleware ทุก request → log เวลา
app.use((req, res, next) => {
  console.log(`[${new Date()}] ${req.method} ${req.url}`);
  next(); // ไป middleware หรือ route ถัดไป
});

// GET / → ตอบ Hello
app.get("/", (req, res) => {
  res.status(200).type('text/plain').send("Hello, Home Page");
});

// GET /about → ตอบ About
app.get("/about", (req, res) => {
  res.status(200).type('text/plain').send("This is the About page");
});

app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

```
[nodemon] starting `node index.js`
```

```
http://localhost:3000
```

```
[Mon Sep 08 2025 21:48:50 GMT+0700 (เวลาอินโดจีน)] GET /
```

GET http://localhost:3000/

Params Auth Headers (6) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body 200 OK

Raw Preview Visualize

1 Hello, Home Page

Middleware แบบ Built-in

- Built-in Middleware คือ middleware ที่ Express เตรียมมาให้แล้วในตัว ไม่ต้องเขียนเอง ไม่ต้องติดตั้งเพิ่ม แค่เรียกใช้จาก express ได้เลย
 1. `express.json()` ใช้ parse ข้อมูล JSON แบบ `application/json` (แปลง request body ที่เป็น JSON ให้อยู่ในรูปแบบ Object ที่อ่านได้จาก `req.body`)
 - ถ้าไม่ใช้ `req.body` จะเป็น `undefined` เวลาส่ง JSON
 - ใช้กับ API ที่ส่ง body เป็น JSON เช่น Fetch, Axios, Postman
 - 2. `express.urlencoded({ extended: true })` ใช้ parse ข้อมูลที่ส่งจากฟอร์ม HTML (แบบ `application/x-www-form-urlencoded`) เหมาะกับการส่งข้อมูลจาก `<form method="POST">`
 - ต้องระบุ `{ extended: true }` หรือ `{ extended: false }`
 - ถ้าเป็นฟอร์มง่าย ๆ ใช้ `extended: false` ถ้าฟอร์มซับซ้อน มี object ซ้อนกัน ใช้ `extended: true`
 - ถ้าไม่ใช้ `req.body` จะเป็น `undefined` เวลาโพสต์จากฟอร์ม



กำหนด route ให้ส่งค่ากลับเป็น json

```
import express from "express";
const app = express();
const PORT = 3000;

// GET / → ตอบ Hello
app.get("/", (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
});

app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

- ปกติสามารถใช้ res.send() ส่งข้อมูลกลับไปเป็นข้อมูล JSON โดยจะต้องกำหนดชนิดข้อมูล res.type("application/json").send({ msg: "hello" });
- แต่ถ้าต้องการส่งข้อมูล JSON และใช้กับ API แนะนำให้ใช้ ใช้ res.json()

GET http://localhost:3000/

Params Auth Headers (6) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body 200 OK

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "date": {
4     "timestamp": "2025-09-09T12:55:27.194Z"
5   }
6 }
```



ทดสอบการส่งข้อมูลแบบ x-www-form-urlencoded

```
import express from "express";
const app = express();
const PORT = 3000;

app.use(express.urlencoded({extended:true}))
app.use(express.json())

app.post("/login", (req, res) => {
  console.log("Headers:", req.headers["content-type"]);
  console.log("Body:", req.body);
  res.send(`Welcome ${req.body.username}`);
});

app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

- `express.urlencoded()` และ `express.json()` เป็น middleware สำเร็จรูปที่ Express เขียนให้แล้ว ไม่ต้องไปใส่ next

application/x-www-form-urlencoded

- เป็น ค่า default ของ `<form method="POST">` ข้อมูลจะถูกเข้ารหัส (encode) เป็น `key=value&key2=value2` คล้าย query string
- ใช้ได้กับข้อมูล text, number, string สั้น ๆ

เข้าสู่ระบบ

Username Password

ผลลัพธ์จะแสดงที่นี่

```
[nodemon] starting node index.js
http://localhost:3000
{ username: 'alice', password: '1234' }
```

POST http://localhost:3000/login

Params Auth Headers (8) **Body** Scripts Settings

x-www-form-urlencoded

☒	username	alice
☒	password	1234
	Key	Value

Body 200 OK

HTML Preview Visualize

1 Welcome alice

API

- API (Application Programming Interface): เป็นชุดของกฎและโปรโตคอลที่อนุญาตให้ส่วนประกอบของซอฟต์แวร์ต่าง ๆ สามารถสื่อสารกันได้ ในบริบทของการพัฒนาเว็บ API มักหมายถึงบริการเว็บที่ให้ข้อมูลหรือฟังก์ชันต่าง ๆ แก่แอปพลิเคชันหรือไคลเอนต์อื่น ๆ (เช่น แอปพลิเคชันบนมือถือหรือเซิร์ฟเวอร์อื่น)
- API มักถูกเข้าถึงผ่าน Route แต่ความแตกต่างที่สำคัญคือ API เน้นที่การให้ข้อมูลหรือฟังก์ชันการทำงาน มากกว่าการแสดงผลหน้าเว็บ
- ยกตัวอย่างเช่น Route อย่าง `/api/v1/users` อาจคืนค่ารายชื่อผู้ใช้ในรูปแบบ JSON ที่แอปพลิเคชันไคลเอนต์สามารถนำไปใช้งานได้



ทดสอบการส่งข้อมูลแบบ REST API

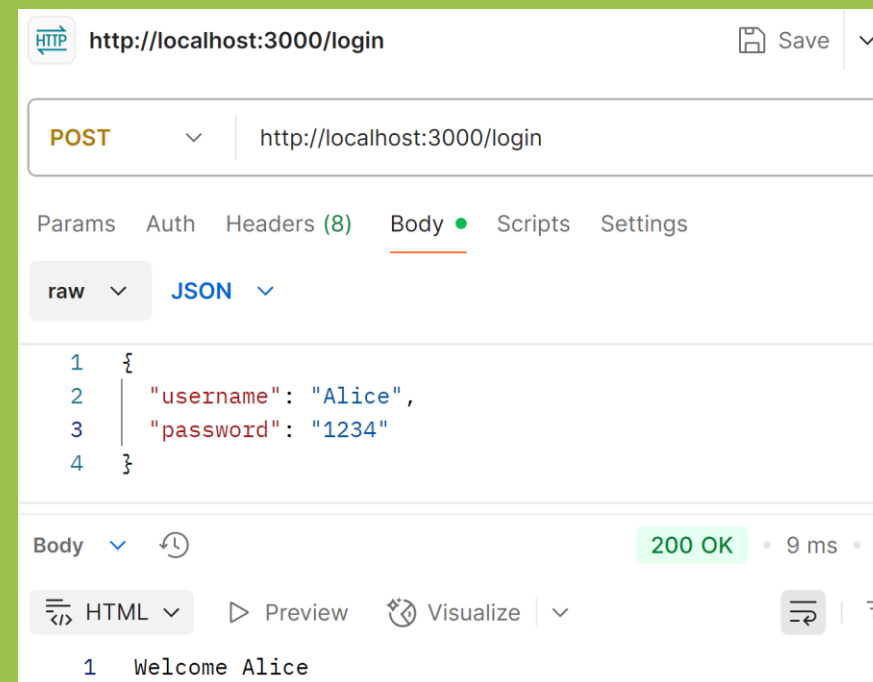
```
import express from "express";
const app = express();
const PORT = 3000;

app.use(express.urlencoded({extended:true}))
app.use(express.json())

app.post("/login", (req, res) => {
  console.log("Headers:", req.headers["content-type"]);
  console.log("Body:", req.body);
  res.send(`Welcome ${req.body.username}`);
});

app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

- `express.urlencoded()` และ `express.json()` เป็น middleware สำเร็จรูปที่ Express เขียนให้แล้ว ไม่ต้องไปใส่ next



```
[nodemon] starting node index.js
http://localhost:3000
{ username: 'alice', password: '1234' }
```



ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (1)

- ก่อนที่จะเริ่มสร้าง route จะสร้างไฟล์ที่เก็บข้อมูลเสื้อผ้า โดยสร้างไฟล์ชื่อว่า product.json และใส่โค้ดของข้อมูลเสื้อผ้า เพื่อจำลองข้อมูล เนื่องจากยังไม่มีฐานข้อมูล
- (ใช้ array แทนฐานข้อมูลจริง)

```
[  
  {  
    "id": "P01",  
    "name": "เสื้อยืด",  
    "color": "สีเหลือง",  
    "price": "300"  
  },  
  {  
    "id": "P02",  
    "name": "เสื้อยืดคอปก",  
    "color": "สีเขียว",  
    "price": "290"  
  },  
  {  
    "id": "P03",  
    "name": "กางเกงขาสั้นทรงบอย",  
    "color": "เข้ม",  
    "price": "590"  
  }  
]
```



ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (2)

- สร้างไฟล์ app.js เพื่อเป็นไฟล์หลัก เพิ่มโค้ดเพื่อสร้าง web server
- ใช้คำสั่ง npm init -y เพื่อสร้างไฟล์ package.json
- เปิดไฟล์ package.json แล้วเพิ่ม "type": "module" และ กำหนดไฟล์เริ่มต้นของโปรเจกต์ที่ "main"
- ติดตั้ง Express.js ได้ผ่านคำสั่ง npm install express
- ติดตั้ง nodemon โดยใช้คำสั่ง npm install nodemon
- เปิด server โดยใช้ npx nodemon index.js

app.js

```
import express from "express";
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})

app.listen(PORT, () =>
  console.log(`http://localhost:${PORT}`));
```

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (3)

- สร้างไฟล์ middleware.js รวม middleware” ไว้ในไฟล์เดียว
- แล้ว import มาใช้ใน app.js จะช่วยให้โค้ดเป็นระเบียบมากขึ้น

```
import express from 'express';  
  
const IndexMiddleware = express();  
  
IndexMiddleware.use(express.urlencoded({extended:true}))  
IndexMiddleware.use(express.json())  
  
export default IndexMiddleware
```

middleware.js

```
import express from "express";  
import IndexMiddleware from "./middlewares.js";  
const app = express();  
const PORT = 3000;  
  
app.use(IndexMiddleware)  
  
app.get('/', (req, res) => {  
  res.status(200).json({  
    success: true,  
    date: {  
      timestamp: new Date()  
    }  
  })  
})  
  
app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

app.js



ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (4)

- ทดลองสร้าง middleware เพื่อ log ข้อมูลต่าง ๆ เกี่ยวกับการร้องขอ
- โดยสร้างไฟล์ logger.js
- แล้ว import ไปใช้ใน middleware.js

logger.js

```
export function logger(req, res, next) {  
  console.log(`[${new Date()}] ${req.method} ${req.url}`);  
  next();  
}
```

middlewares.js

```
import express from 'express';  
import { logger } from './logger.js';  
const IndexMiddleware = express();  
  
IndexMiddleware.use(express.urlencoded({extended:true}))  
IndexMiddleware.use(express.json())  
IndexMiddleware.use(logger)  
export default IndexMiddleware
```

```
[nodemon] starting `node app.js`  
http://localhost:3000  
[Tue Sep 09 2025 21:48:22 GMT+0700 (เวลาอินโดจีน)] GET /
```



ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (5)

- สร้าง route เก็บไว้ที่ route.js
- โดยจะอ่านข้อมูลจากไฟล์ product.json เข้ามาใช้งานด้วยคำสั่ง import data from './product.json' with { type: 'json' };
- กำหนดเส้นทาง (Route) แบบ GET ที่ path /product ให้ ตั้งค่า HTTP status code เป็น 200 OK และส่งข้อมูลเป็น json
- เมื่อ client เรียก GET http://localhost:3000/product ทำให้ server ตอบกลับด้วยข้อมูลแบบ JSON
- แล้ว import route.js ไปใช้ใน app.js

```
import express from 'express'
import data from './product.json' with { type: 'json' };
const IndexRouter = express();

IndexRouter.get('/product', (req, res) => {
  res.status(200).json(data)
})

export default IndexRouter
```

```
import express from "express";
import IndexMiddleware from "./middlewares.js";
import IndexRouter from "./route.js";
const app = express();
const PORT = 3000;

app.use(IndexMiddleware)
app.use(IndexRouter);
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})
```

```
app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

route.js

app.js

GET http://localhost:3000/product

Params

Query Params

Key	Value	De...
-----	-------	-------

Body

200 OK • 36 ms • 5

JSON

```
1 [
2   {
3     "id": "P01",
4     "name": "เสื้อยืด",
5     "color": "สีเหลือง",
6     "price": "300"
7   },
8 ]
```

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (6)

- เพิ่มโค้ดที่ route.js
- เพิ่ม กำหนดเส้นทาง (Route) แบบ GET ที่ path /product/:id ให้ค้นหาข้อมูล id ที่ได้รับในไฟล์ product.js แล้วแสดงข้อมูลที่พบ
- .find(...) ใช้ค้นหา element ตัวแรก ที่ตรงกับเงื่อนไขที่กำหนด
 - ถ้าเจอ คืนค่า element นั้น
 - ถ้าไม่เจอ คืนค่า undefined
- data => data.id === req.params.id
 - เป็น arrow function ที่กำหนดเงื่อนไขการค้นหาโดยเชื่อว่า data.id เท่ากับ req.params.id ใหม่
 - data (คือ element แต่ละตัวใน array)
 - req.params.id คือค่าพารามิเตอร์ที่มาจาก URL (เช่น /products/P02) → req.params.id = "P02")

```

IndexRouter.get('/product/:id', (req, res) => {
  const fdata = data.find(data => data.id === req.params.id);
  if (!fdata) return res.status(404).send('Product not found');
  res.status(200).json(fdata);
})

```

route.js

GET http://localhost:3000/product/P02

Params

Query Params

	Key	Value	De...
	Key	Value	De...

Body 200 OK • 21 ms

{ } JSON

```

1 {
2   "id": "P02",
3   "name": "เสื้อยืดคอปก",
4   "color": "สีเขียวก",
5   "price": "290"
6 }

```

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (7)

- เพิ่มโค้ดที่ route.js
- เพิ่ม กำหนดเส้นทาง (Route) แบบ POST ที่ path /product เพื่อเพิ่มข้อมูลในไฟล์ product.js (เป็นการเพิ่มข้อมูลชั่วคราว)
- .push() เป็น เมธอดของ Array ใช้สำหรับ เพิ่ม element ใหม่เข้าไปท้าย array คำนวณค่าเป็น ความยาวใหม่ของ array

route.js

```
IndexRouter.post('/product', (req, res) => {
  const product_data = req.body;
  data.push( product_data)
  res.status(201).json(product_data)
})
```

POST http://localhost:3000/product/

Params Auth Headers (8) Body Scripts Settings

raw JSON

```
1 {
2   "id": "P04",
3   "name": "กางเกงขาสั้น",
4   "color": "สีดำ",
5   "price": "490"
6 }
```

Body 201 Created

JSON Preview Visualize

```
1 {
2   "id": "P04",
3   "name": "กางเกงขาสั้น",
4   "color": "สีดำ",
5   "price": "490"
6 }
```

POST http://localhost:3000/product/

Params Auth Headers (8) Body Scripts Settings

x-www-form-urlencoded

	Key	Value
☒	id	P04
☒	name	กางเกงขาสั้น
☒	color	สีดำ
☒	price	490

Body 201 Created

JSON Preview Visualize

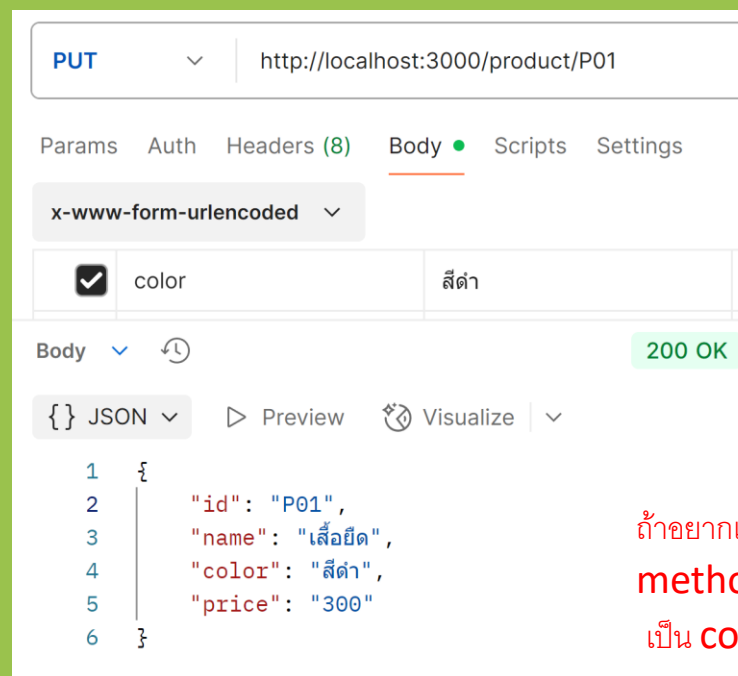
```
1 {
2   "id": "P04",
3   "name": "กางเกงขาสั้น",
4   "color": "สีดำ",
```

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (8)

route.js

- เพิ่มโค้ดที่ route.js
- สร้าง API สำหรับอัปเดตข้อมูลเสื้อผ้าตาม id อัปเดต ข้อมูลเสื้อผ้า ซึ่งปกติแล้วจะทำผ่าน method put โดยจะต้องเลือกที่จะอัปเดตจาก id ที่เท่าไร พร้อมส่งข้อมูลที่จะอัปเดตผ่าน request body
- .findIndex() เป็นเมธอดของ Array ทำงานคล้าย .find() แต่ คืนค่าเป็น index (ตำแหน่งใน array) ถ้าไม่เจอ จะคืนค่า -1
- Object.assign(target, source) เป็นเมธอดของ JavaScript สำหรับ คัดลอก property จาก object (source) ไปยังอีก object (target)
 - `|| {}` ถ้า req.body เป็น undefined (เช่น client ไม่ส่งข้อมูลมา) → ใช้ `{}` แทน เพื่อป้องกัน error

```
IndexRouter.put('/product/:id', (req, res) => {
  const updateIndex = data.findIndex(item => item.id === req.params.id);
  if (updateIndex === -1) return res.status(404).send('Product not found');
  const updatedData = Object.assign(data[updateIndex], req.body || {});
  res.status(200).json(updatedData);
})
```



ถ้าอยากเช็คข้อมูลอัปเดตใหม่ไปที่ `/product` ด้วย method get ที่ id: P01 จะเปลี่ยน color: สีเหลือง เป็น color: สีดำ

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (9)

route.js

- สร้าง API สำหรับลบข้อมูลเสื้อผ้า
- การลบข้อมูลเสื้อผ้า จะทำผ่าน method delete และส่ง id ของข้อมูลเสื้อผ้าที่ต้องการจะลบไป

```
IndexRouter.delete('/product/:id', (req, res) => {  
  const deletedIndex = data.findIndex(data => data.id === req.params.id)  
  data.splice(deletedIndex, 1) //ลบข้อมูลโดยตำแหน่งเริ่มต้นที่จะลบ (deletedIndex) ลบจำนวน 1 รายการ  
  res.status(204).send()  
})
```

DELETE ▼ http://localhost:3000/product/P01

Params Auth Headers (6) Body ● Scripts Settings

x-www-form-urlencoded ▼

☐	color	สีดำ
--------------------------	-------	------

Body ▼ 🕒 204 No Content

Raw ▼ ▶ Preview 🔍 Visualize ▼

1

ถ้าสมมติจะสร้างร้านขายเสื้อผ้า (10)

- สามารถสร้าง controller.js เพื่อสร้าง ฟังก์ชันที่ใช้ดำเนินการในแต่ละ route
- เช่น
- `const Controller = { ... }`
 - อธิบายชื่อ Controller ด้านในมี property หลายตัว ซึ่งเป็นฟังก์ชัน (method)
- `getProduct: (req, res) => { ... }` เป็นฟังก์ชันที่ใช้จัดการ Request / Response
- เมื่อสร้างตัวดำเนินการของ route ให้นำไปใช้งาน route.js

route.js

```
import express from 'express'
import data from './product.json' with { type: 'json' };
const IndexRouter = express();

IndexRouter.get('/product', (req, res) => {
  res.status(200).json(data)
})
```

controller.js

```
import data from './product.json' with { type: 'json' };

const getProduct = (req, res) => {
  res.status(200).json(data)
}

export {getProduct}
```

route.js

```
import express from 'express'
import data from './product.json' with { type: 'json' };
import {getProduct} from './controller.js'
const IndexRouter = express();

IndexRouter.get('/product', getProduct)
```

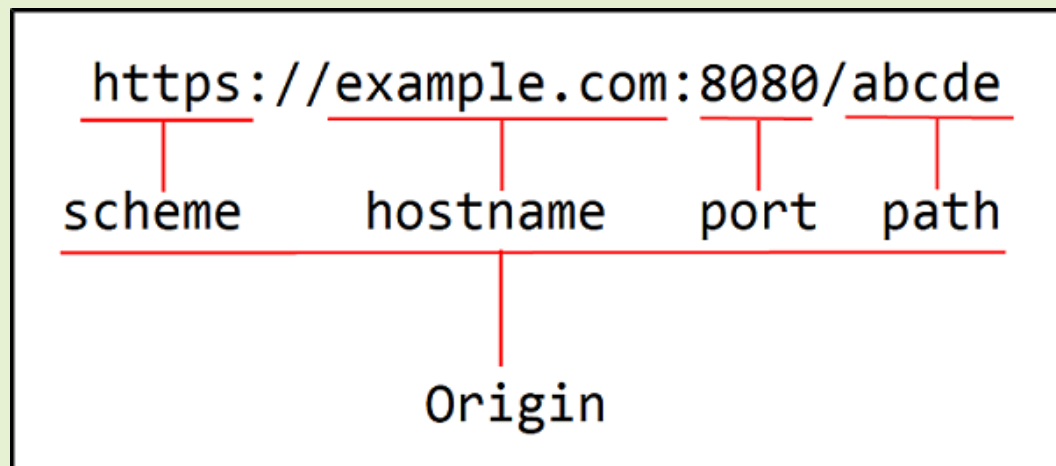
CORS ?

- อินเทอร์เน็ตเป็นการสื่อสารระหว่างคอมพิวเตอร์ ดังนั้นคอมพิวเตอร์แต่ละเครื่องต้องมี Protocol ที่เหมือนกัน ถึงจะสื่อสารกันรู้เรื่อง เว็บเบราว์เซอร์จะส่ง HTTP request เมื่อต้องการขอข้อมูลข้ามโดเมนหรือ port ที่ต่างกัน และต้องทำตามข้อตกลงการสื่อสาร (Protocol)
- CORS ย่อมาจาก Cross-Origin Resource Sharing คือการแชร์ทรัพยากรหรือดึงข้อมูลจาก Origin (เว็บเซิร์ฟเวอร์) อื่นได้ โดยปกติแล้วเว็บแอปจะดึงข้อมูลจากเซิร์ฟเวอร์เดียวกันเท่านั้น
- เป็นกลไกที่ใช้ HTTP headers เพิ่มเติมเพื่อให้เบราว์เซอร์ได้รับสิทธิ์ในการเข้าถึงทรัพยากรที่เลือกจากเซิร์ฟเวอร์บนโดเมนอื่นมาแสดงบนหน้าเว็บเบราว์เซอร์ได้
- CORS จะกำหนดวิธีการที่เว็บเบราว์เซอร์และเซิร์ฟเวอร์สามารถโต้ตอบเพื่อกำหนดว่าจะให้อนุญาตในการขอข้อมูลข้ามโดเมนหรือไม่



Origin ?

- Origin คือ headers ที่บอกว่า Request ที่ระบุ URL ของไคลเอนต์ที่ส่งคำขอ
- Origin ประกอบด้วย
 1. Scheme เช่น http, https
 2. Hostname เช่น localhost, www.abc.com, abc.com
 3. Port เช่น 3000, 5000, 8000, 8080



Origin เหล่านี้ต่างกันหรือไม่?

http://abc.com

http://abc.com

http://abc.com:3000

http://abc.com

https://abc.com

http://www.abc.com

http://abc.com:3001

http://abc.com:80

=> Scheme ต่างกันร้องขอข้อมูลไปคนละที่

=> Hostname ต่างกันร้องขอข้อมูลไปคนละที่

=> Port ต่างกันร้องขอข้อมูลไปคนละที่

=> ทุกเว็บถ้าไม่ได้ port จะรันอยู่ที่ port 80 ดังนั้นมันคือ Origin เดียวกัน



SAME-ORIGIN Policy

domain-a.com



REQUEST



domain-a.com



domain-a.com



REQUEST

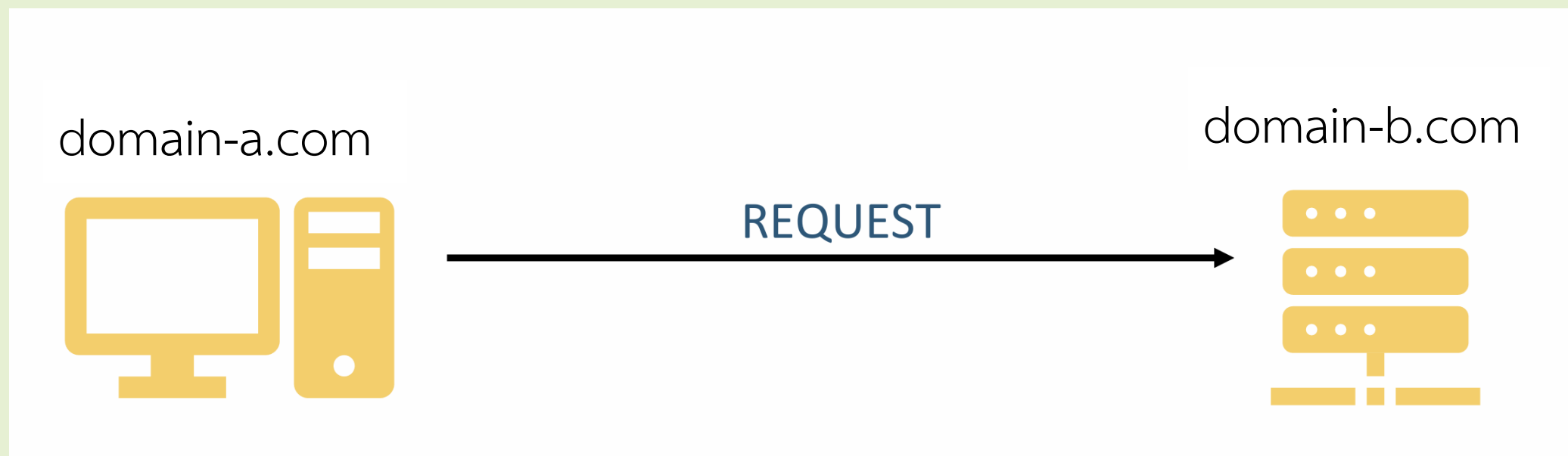


domain-b.com



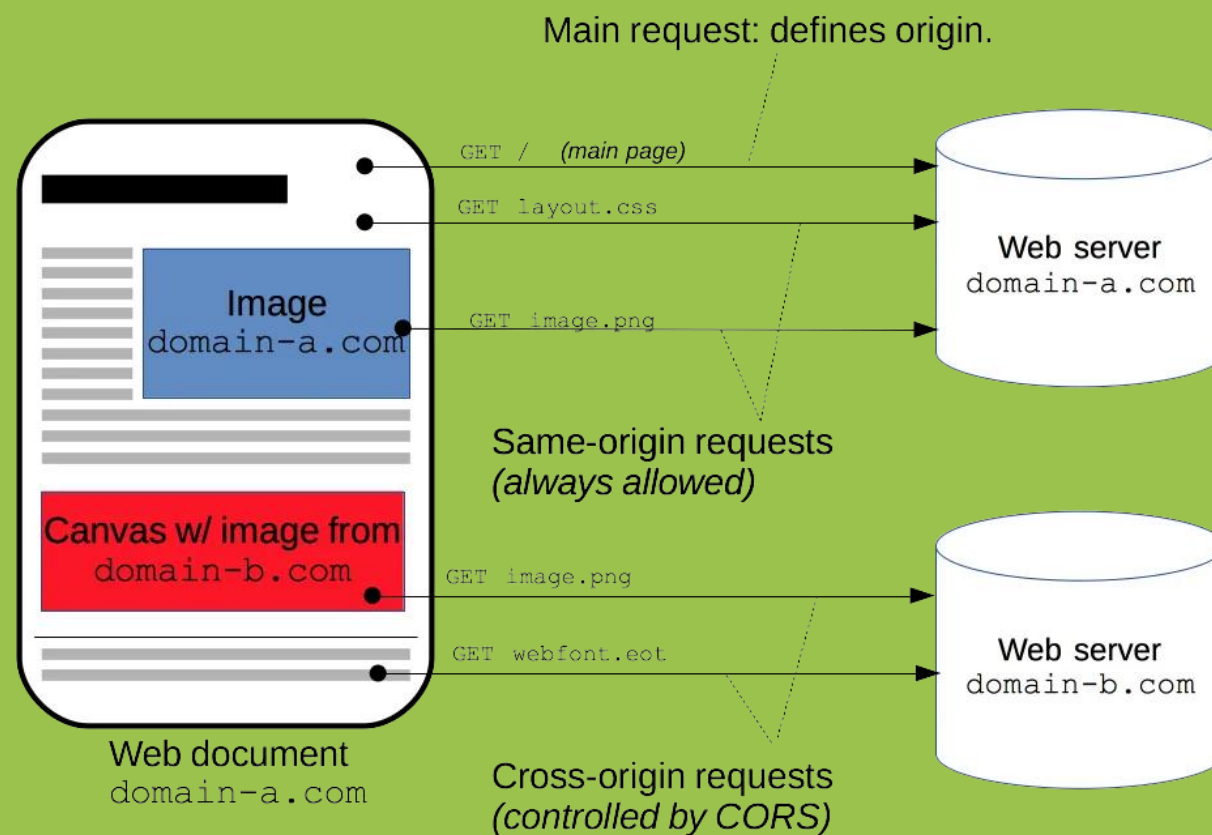
CORS จะเกิดขึ้นเมื่อใด?

- CORS จะเกิดขึ้นเมื่อ Origin ของผู้รับและผู้ส่งต่างกัน



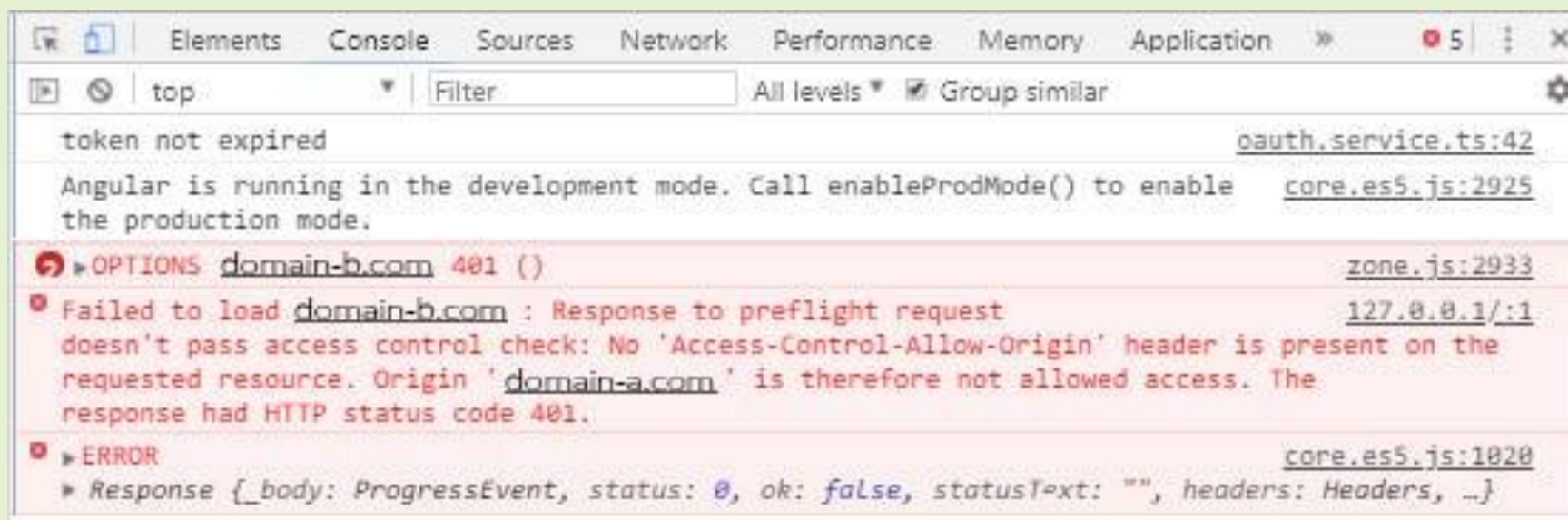
CORS

- ปัจจุบันมักจะแยกฝั่ง Front-end และ Back-end ออกจากกัน โดยแยกใช้คนละโดเมน ด้วยเหตุผลเรื่องความปลอดภัยของ Browsers HTTP การอนุญาตให้เข้าถึงแหล่งข้อมูลจะต้องอยู่โดเมนเดียวกันเท่านั้น เว้นแต่ว่าแหล่งข้อมูลนั้นจะอนุญาตให้โดเมนของ Browsers สามารถเข้าถึงข้อมูลเหล่านั้นได้



https://mdn.mozillademos.org/files/14295/CORS_principle.png

ตัวอย่าง Error เมื่อไม่มีการกำหนด CORS



ตัวอย่าง โดเมน domain-a.com ไม่ได้รับอนุญาตจาก domain-b.com หากต้องการ
ร้องขอข้อมูลจาก domain-b.com ก็จะเจอ Error แบบนี้

วิธีกำหนด CORS แบบอนุญาตทั้งหมด

- ใน Node.js สามารถติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- เรียกใช้งาน CORS ด้วยคำสั่ง `import cors from "cors";`

```
import express from "express";
import cors from "cors";
const app = express();
const PORT = 3000;

// เปิด CORS ทุก origin จัดการ header ให้ทุก request
app.use(cors());

// Middleware สำหรับ parse body
app.use(express.urlencoded({extended:true}))
app.use(express.json())

app.get('/',(req,res)=>{
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})
app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

ตัวอย่างการใช้ CORS middleware แบบอนุญาตทั้งหมด

เช่น ใช้กับเว็บร้านค้า เพราะไม่สามารถระบุ โดเมน ของ client ได้

OPTIONS

http://localhost:3000/

Send

Params

Auth

Headers (8)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value		Bulk Edit	Pr
☒	Origin	http://localhost:5173			
☒	Access-Control-Request-...	GET			
	Key	Value	Description		

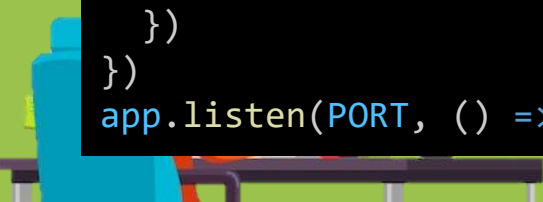
Headers

204 No Content

17 ms

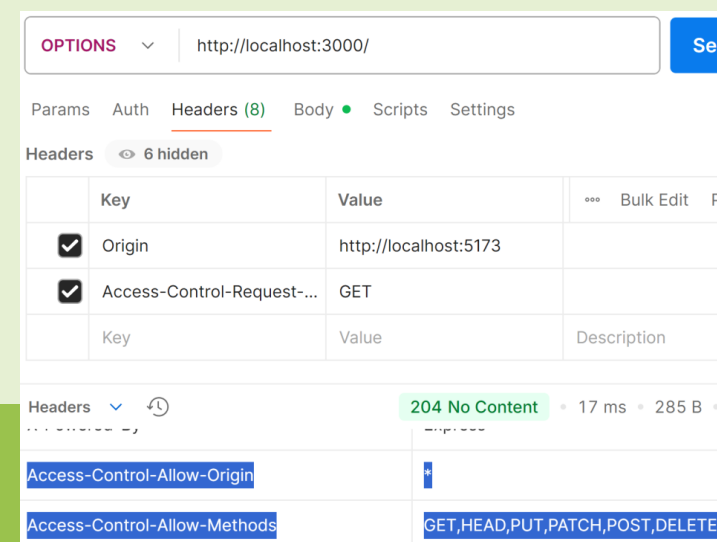
285 B

Access-Control-Allow-Origin	*
Access-Control-Allow-Methods	GET,HEAD,PUT,PATCH,POST,DELETE



ทดสอบ CORS ด้วย Postman

- สามารถจำลอง preflight request (OPTIONS) ที่ browser ใช้ตรวจสอบสิทธิ์ CORS ได้
- Preflight Request เป็น request แบบพิเศษ (OPTIONS) ที่ Browser ส่งไปยัง Server ก่อน request จริง
 - มีไว้เพื่อถามว่า “Server อนุญาตไหม ถ้าเราจะส่ง request ข้าม origin (Cross-Origin) ด้วย method/headers แบบนี้?”
 - ถ้า Server ตอบว่าอนุญาต แล้ว Browser จะส่ง request จริงต่อ
 - ถ้า Server ไม่อนุญาต แล้ว Browser จะ block ทันที (Frontend จะเจอ CORS error)
- ขั้นตอนใน Postman
 - เปิด Postman → กด New request
 - เลือก Method: OPTIONS
 - ใส่ URL เช่น <http://localhost:3000/product>
 - ไปที่แท็บ Headers แล้วเพิ่ม:
 - Origin: `http://localhost:5173`
 - Access-Control-Request-Method: GET
 - ** (ถ้าอยากทดสอบ POST, PUT → ใส่เป็น POST หรือ PUT)
 - กด Send
- Postman ไม่เจอ CORS error (เป็นเรื่องของ browser เท่านั้น)
- แต่สามารถ จำลอง preflight (OPTIONS request) ใน Postman เพื่อดูว่า server ตอบ CORS headers กลับมาถูกต้องหรือไม่
- ถ้าเห็น Access-Control-Allow-Origin ตรงกับ Origin → แสดงว่า server เปิด CORS เรียบร้อย



วิธีกำหนด CORS แบบกำหนดโดเมน

- ใน Node.js สามารถติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- เรียกใช้งาน CORS ด้วยคำสั่ง `const cors = require("cors");`

```
import express from "express";
import cors from "cors";
const app = express();
const PORT = 3000;
// เปิด CORS แบบกำหนดโดเมน
app.use(cors({ origin: "http://127.0.0.1:5500" }));

// Middleware สำหรับ parse body
app.use(express.urlencoded({extended:true}))
app.use(express.json())

app.get('/',(req,res)=>{
  res.status(200).json({
    success: true,
    date: {
      timestamp: new Date()
    }
  })
})
app.listen(PORT, () => console.log(`http://localhost:${PORT}`));
```

ตัวอย่างการใช้ CORS แบบกำหนดโดเมน

OPTIONS http://localhost:3000/ Send

Params Auth Headers (8) Body Scripts Settings

Headers 6 hidden

	Key	Value	...	Bulk Edit	Pres...
☒	Origin	http://localhost:5173			
☒	Access-Control-Request-...	GET			
	Key	Value	Description		

Headers 204 No Content • 9 ms • 313 B •

Access-Control-Allow-Origin	http://127.0.0.1:5500
Vary	Origin, Access-Control-Request-Headers

ใช้หน้าเว็บทดสอบ cors

- ใช้ฟังก์ชัน `JSON.stringify()` ของ JavaScript เพื่อแปลง Object / Array ให้กลายเป็นข้อความ (string)
- โครงสร้างคำสั่ง `JSON.stringify(value, replacer, space)`
 - `value` = ค่าที่จะเอามาแปลง (object, array, number, string ฯลฯ)
 - `replacer` = ตัวเลือก (ไม่ใส่ก็ได้) ถ้าอยากเลือกที่จะเอา field ไหนบ้าง
 - `space` = ตัวเลขบอกว่าจะจัด indent (ย่อหน้า) กี่ช่อง

ตัวอย่าง Fetch

ดึงข้อมูล

เกิดข้อผิดพลาด: Failed to fetch

```
<!doctype html>
<html lang="th">
<head>
  <meta charset="utf-8" />
  <title>Fetch ตัวอย่างสั้นๆ</title>
  <meta name="viewport" content="width=device-width,initial-scale=1" />
</head>
<body>
  <h1>ตัวอย่าง Fetch</h1>
  <button id="btn">ดึงข้อมูล</button>
  <p id="out">รอผลลัพธ์...</p>
  <script>
    const BTN = document.getElementById('btn');
    const OUT = document.getElementById('out');
    BTN.addEventListener('click', async () => {
      OUT.textContent = 'กำลังดึงข้อมูล...';
      try {
        const res = await fetch('http://localhost:3000/');
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        OUT.innerText = JSON.stringify(data, null, 2);
      } catch (err) {
        OUT.innerText = 'เกิดข้อผิดพลาด: ' + err.message;
      }
    });
  </script>
</body>
</html>
```

```
✖ Access to fetch at 'http://localhost:3000/' from origin 'null' has been blocked by CORS policy: The 'Access-Control-Allow-Origin' header has a value 'http://127.0.0.1:5500' that is not equal to the supplied origin. Have the server send the header with a valid value.
✖ ▶ GET http://localhost:3000/ net::ERR_FAILED 200 (OK) home.html:20 4
```

เขียนหน้าเว็บร้องขอข้อมูลอย่างไร

เข้าสู่ระบบ

Username Password

ผลลัพธ์จะแสดงที่นี่

```
app.post("/login", (req, res) => {  
  console.log("Headers:", req.headers["content-type"]);  
  console.log("Body:", req.body);  
  const { username, password } = req.body || {};  
  if (!username || !password) {  
    return res.status(400).json({ error: "กรุณากรอก username และ password" });  
  }  
  res.json({ message: "รับข้อมูลเรียบร้อยแล้ว", username, lengthOfPassword: password.length });  
});
```



ส่งแบบ JSON

```
<body>
  <h1>เข้าสู่ระบบ</h1>
  <form id="loginForm">
    <label>Username <input id="username" name="username" type="text" required /></label>
    <label> Password<input id="password" name="password" type="password" required /></label>
    <button type="submit">ส่งข้อมูล</button>
  </form>
  <div id="result"><i>ผลลัพธ์จะแสดงที่นี่</i></div>
  <script>
    const form = document.getElementById('loginForm');
    const resultBox = document.getElementById('result');
    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      resultBox.innerHTML = 'กำลังส่ง...';
      const payload = {
        username: document.getElementById('username').value.trim(),
        password: document.getElementById('password').value
      };
      try {
        const res = await fetch('http://localhost:3000/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(payload)
        });
        // แปลงผลลัพธ์เป็น JSON แล้วแสดง
        const data = await res.json();
        if (!res.ok) throw new Error(data?.error || `HTTP ${res.status}`);
        resultBox.innerHTML = JSON.stringify(data, null, 2);
      } catch (err) {
        resultBox.innerHTML = 'เกิดข้อผิดพลาด: ' + err.message;
      }
    });
  </script>
</body>
```



ส่งแบบ x-www-form-urlencoded

- แก้ไขโค้ดในส่วนไฮไลน์สีเหลืองที่ส่งแบบ JSON สไลด์หน้า 40 เป็นโค้ดด้านล่าง

```
const formData = new URLSearchParams(payload); //สร้าง query string จาก object เป็น "username=ค่า&password=ค่า"
const res = await fetch('http://localhost:3000/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: formData.toString()
});
```



ใบงาน

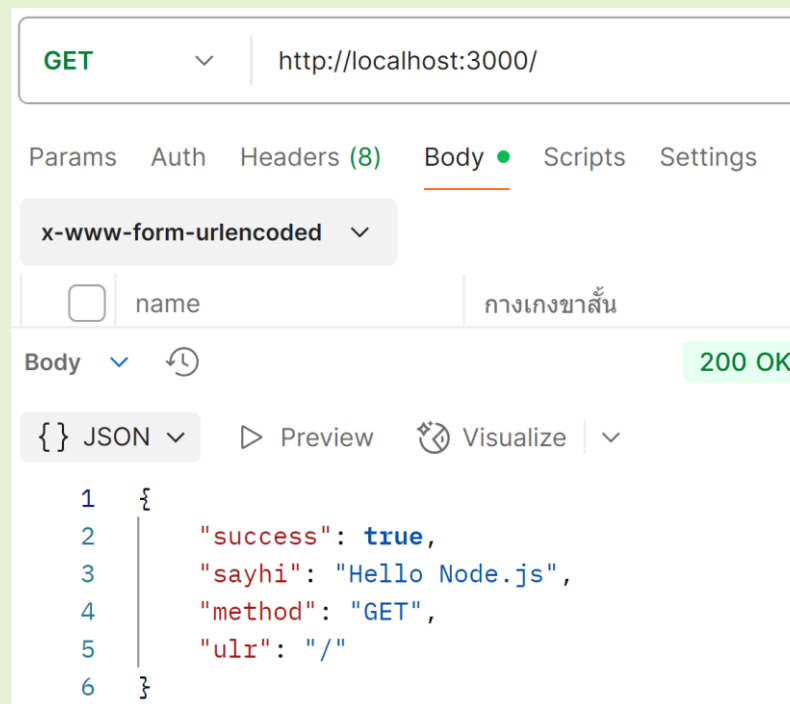
- สร้างโฟลเดอร์ lab8_รหัสของนักศึกษา เพื่อใช้เก็บโปรเจกต์ซึ่งภายในประกอบไปด้วยโฟลเดอร์ดังนี้
- myapp/
 - |
 - |—— package.json # เก็บ dependencies และ script
 - |—— app.js # main app (ตั้งค่า express, middleware, routes)
 - |—— middlewares.js. #รวมการตั้งค่า middleware ทั้งหมด
 - |—— log.js. #สร้าง middleware สำหรับ log ข้อมูลของ request ที่เข้ามาทั้งหมด
 - |—— book.route.js # ไฟล์ route
 - |—— book.controller.js # ฟังก์ชันของแต่ละ route
 - |—— book-data.json # ข้อมูลจำลอง (database ชั่วคราว)
 - |—— index.html # เก็บไฟล์หน้าเว็บ (HTML, CSS, JS) ทดสอบ fetch



ใบงาน

- 1. ในการรันจะแสดงผลลัพธ์เป็นข้อความว่า “<http://localhost:3000>” (3000 คือการตั้งค่าของ PORT ที่กำหนดในโค้ด) มีการกำหนด status-code เป็น 200 และเมื่อ ใส่ URL : <http://localhost:3000> จะแสดงรูป

```
[nodemon] starting `node app.js`  
http://localhost:3000
```

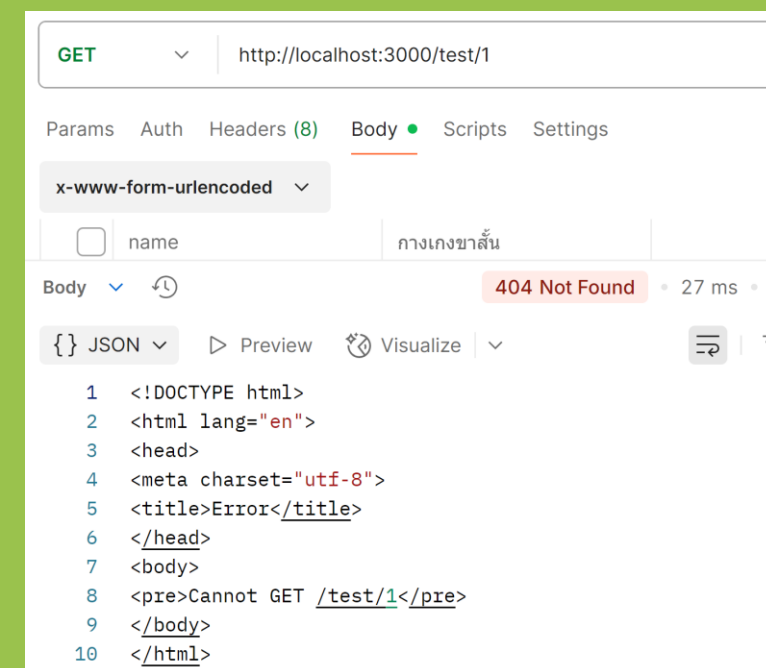


ใบงาน

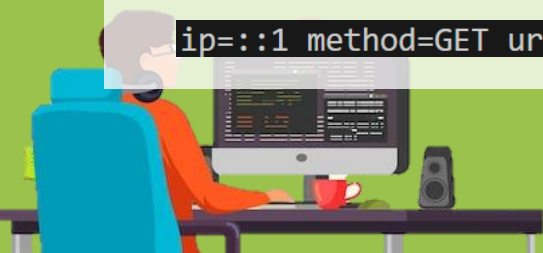
- 2. สร้างไฟล์ books.json เก็บไว้ที่โฟลเดอร์ mock-data ภายในสร้างข้อมูลตามตารางด้านล่าง

id	title	author
1	The Hunger Games	Harper Lee
2	The Da Vinci Code	Dan Brown
3	The Lord of the Rings: The Fellowship of the Ring	J.R.R. Tolkien

- 3. สร้างไฟล์ log.js เป็น Middleware เพื่อแสดงข้อมูลเกี่ยวกับคำขอ (request) ที่เข้ามา ได้แก่ ip จากคำสั่ง req.ip, URL, วิธีการที่ใช้ (HTTP method), วันเวลา
 - เช่น เมื่อใช้ `http://localhost:3000/test/1` โดยใช้ get ที่ postman ส่ง Request เข้ามา แสดงดังรูป



```
ip:::1 method=GET url=/test/1 day=[Wed Sep 10 2025 21:53:43 GMT+0700 (เวลาอินโดจีน)]
```



ใบงาน

- 4. สร้างไฟล์ `book.route.js` เขียนโค้ดเพื่อสร้าง router เพื่อ ดึง อัปเดต เพิ่ม ลบข้อมูลหนังสือ โดยเรียกใช้ฟังก์ชันที่อยู่ในไฟล์ `book.controller.js` ภายในการกำหนด router ดังนี้
 - 1. ดึงข้อมูลรายการหนังสือทุกรายการ ด้วยเมธอด GET ผ่าน URL: `http://localhost:3000/api/book`
 - 2. ดึงข้อมูลหนังสือตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด GET ตัวอย่าง URL คือ `http://localhost:3000/book/1` โดย 1 คือ id ของหนังสือ
 - 3. เพิ่มข้อมูลหนังสือ โดยรับข้อมูล id, title, author จาก request แล้ว response ข้อมูลที่สร้างด้วย status-code เป็น 201 ด้วยเมธอด POST ผ่าน URL: `http://localhost:3000/book`
 - 4. แก้ไขข้อมูลหนังสือตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด PUT ตัวอย่าง URL : `http://localhost:3000/book/3` โดย 3 คือ id ของหนังสือ แล้ว response ข้อมูลที่แก้ไขด้วย status-code เป็น 200
 - 5. ลบข้อมูลหนังสือ ตาม id ที่กำหนดที่ Parameter ใน URL ด้วยเมธอด delete ตัวอย่าง URL : `http://localhost:3000/book/3` โดย 3 คือ id ของหนังสือ แล้ว response ด้วย status-code เป็น 204



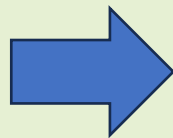
ใบงาน

- 5. สร้างหน้า index.html ดึงข้อมูลหนังสือทั้งหมดออกมาแสดง (ดึงข้อมูลรายการหนังสือทุกรายการ ด้วยเมธอด GET ผ่าน URL: <http://localhost:3000/api/book>)

ข้อมูลหนังสือทั้งหมด

ดึงข้อมูล

รอผลลัพธ์...



ข้อมูลหนังสือทั้งหมด

ดึงข้อมูล

```
[
  {
    "id": "1",
    "title": "The Hunger Games",
    "author": "Harper Lee"
  },
  {
    "id": "2",
    "title": "The Da Vinci Code",
    "author": "Dan Brown"
  },
  {
    "id": "3",
    "title": "The Lord of the Rings: The Fellowship of the Ring",
    "author": "J.R.R. Tolkien"
  }
]
```

